

Adaptive Manipulability-Based Path Planning Strategy for Industrial Robot Manipulators

Henghua Shen , Member, IEEE, Wen-Fang Xie , Senior Member, IEEE, Jianyu Tang, and Tao Zhou

Abstract—In this article, a novel manipulability-based optimal rapidly exploring random tree (RRT*) path planning strategy is proposed for industrial robot manipulators. When sampling in the search space, two constraints, namely, path length and manipulability measure, are imposed to find a minimal-cost path connecting the start and goal points. By tracking the generated path, a robot manipulator's end-effector can traverse the workspace with a shorter length and, meanwhile, avoid configuration singularities. A constrained closed-loop inverse kinematics technique is utilized to exploit the kinematic redundancy to assign a higher manipulability to an end-effector position. Additionally, the metrics of path length and manipulability measure are used to determine the adaptive step size for the RRT* planner. This helps the space-filling tree to grow efficiently toward unsearched areas and find an optimal path. Simulation analysis and experimental results of a six-degree-of-freedom FANUC-M-20iA industrial robot illustrate the efficiency of the proposed path planning methods.

Index Terms—Adaptive step size, manipulability, path planning, rapidly-exploring random tree (RRT*), robot manipulator.

I. INTRODUCTION

ROBOT manipulators have been extensively used in the industrial setting for tasks that are repetitive or exceed the speeds and accuracies of human operators. In many specialized robotic applications, a certain level of autonomy is required. For example, automatically searching for a optimal path from a current pose to a goal pose, which is usually termed as *path planning*. Finding a successful path helps a robot implement tasks with high-quality performance [1], [2].

Existing path planning algorithms can generally be classified into two categories: 1) classical methods and 2) heuristic methods. Heuristic methods, such as neural network (NN) and reinforcement learning (RL) have become popular due to

their advantages, such as learning ability and parallel processing [3]. In the classical path planning methods, two popular path planning algorithms are 1) graph-based algorithm and 2) sample-based algorithm. Graph-based search algorithm relies on the successful mapping of configuration space. However, it suffers from high computational complexity, especially for high-dimensional searching space. Sample-based algorithm has attracted considerable attention because they can efficiently explore a nonconvex and high-dimensional space. For example, the rapidly exploring random tree (RRT) method developed by Steven M. LaValle and James J. Kuffner Jr. [4], [5] is one of the most popular sample-based methods and it has been successfully used in many applications of robot manipulators [6], [7].

However, as pointed out in [8], the RRT method only addresses the feasibility problem. Thus, it is desirable to find improvements of the RRT that converges to an optimal solution, such as the RRT* method. Then, more RRT*-based methods have been proposed to extend the ability of the original RRT* method. For example, the RRT*-Smart method for accelerating the convergence rate of RRT*, the RRT* FND method for path planning in dynamic environments, and the A*-RRT* for avoiding hazardous areas and preferring low-risk routes. By addressing the optimality problem, the RRT* method can take into account multiple constraints imposed by the environments, the human operators, and/or the robots. Therefore, in this article, we use the RRT* method to obtain the autonomy for finding a feasible path with minimal cost.

A great deal of the RRT*-based methods seek to find the shortest path [9], [10], [11]. Therefore, their cost functions are given in terms of distance in various form (such as the Euclidean distance). However, for robot manipulators, a shortest path may contain kinematic singularities. Robot manipulators at the singular configurations can suffer from infeasible joint velocity commands and loss of (or significantly reduced) motion capability. Fortunately, a robot's ability of maneuvering in the workspace can be easily characterized a metric called *manipulability measure* [12]. Therefore, to realize the singularity avoidance in path planning, the manipulability measure can be used in the cost function to optimize the trajectory. For example, Marić et al. [13] discusses the manipulability maximization in the path planning in configuration-space (C-space). However, in many applications, the task information is given in operational space (O-space), such as predetermined end-effector trajectories. Furthermore, for redundant robot manipulators, one end-effector position may have different values of manipulability as mapping from the O-space to the C-space

Manuscript received 6 September 2022; accepted 10 December 2022. Date of publication 6 January 2023; date of current version 16 June 2023. Recommended by Technical Editor Y. Gu and Senior Editor N. G. Tsagarakis. This work was supported by the Horizon Postdoctoral Fellowship Grant at Concordia University. (Corresponding author: Wen-Fang Xie.)

The authors are with the Department of Mechanical, Industrial and Aerospace Engineering, Concordia University, Montreal, QC H3G 1M8, Canada (e-mail: henghua.shen@concordia.ca; wfxie@encs.concordia.ca; jianyu.tang@concordia.ca; tao.zhou@mail.concordia.ca).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TMECH.2022.3231467>.

Digital Object Identifier 10.1109/TMECH.2022.3231467

is not unique, making it challenging to use manipulability as a constraint in the RRT* approaches in O-space. One of the solutions is given in [14], where manipulability approximation of a given pose in the workspace is used. The manipulability approximation is defined either by the average manipulability or the bound of a subset of manipulability data within a voxel. However, the approximation may not accurately represent the actual manipulability of the according configuration. In this article, we aim at finding a solution that can uniquely assign an actual O-space manipulability to each end-effector position by using the constrained closed-loop inverse kinematics.

Another issue addressed in this article is the searching efficiency of RRT* methods. Since RRT* methods are based on a uniform sampling strategy, there is a conflict to simultaneously achieve low computational cost and high space-filling efficiency. In other words, given a larger number of samples, the workspace can be extensively explored, but it takes a longer time to implement the exploration. To improve the exploring efficiency, a great deal of research on nonuniform sampling distribution has been carried out [15], [16], [17]. By purposely sampling in the interesting regions, the planners are more likely to select a desired set of waypoints out of the sampled data. Another effective way to manage the sample distribution is by adaptively changing the step size. For example, in [18] the step size is determined by the distance from the current robot position to the obstacles. Specifically, when the searching nodes get close to obstacles, the step size is reduced to get a fine growth of the RRT tree that helps avoid collisions. On the other hand, when searching in obstacle-free regions, the step size increases to speed up the exploration into unsearched regions. Methods with the similar idea can be found in [19]. However, the RRT* method for singularity avoidance by using the adaptive step size subject to manipulability measure has been given less attention.

The recent literature that is the most relevant to the proposed article includes [18], [19], [20], [21], [22]. Compared to these works, the main contributions of this article are as follows.

- 1) The RRT* step size proposed in this article is adaptive and determined by negative gradient descent of the artificial potential fields. Compared to [18] and [19], where the adaptive RRT* step size is designed for obstacle avoidance for mobile robots, in this article, the step size of the RRT* planner is adaptive to singularities and the goal position for robot manipulators. The challenges to obtain the gradient descent of manipulability measure and Euclidean distance with respect to joint state have been well solved.
- 2) The rationale of the manipulability-based sampling is adopted from [20] and [22]. Compared to [20] and [22], where the manipulability measure is exploited to construct a map (i.e., the probabilistic roadmap [20] or capability map [22]), in this article, we focus on using the metric of manipulability measure in the RRT* path planning strategy so that the generated reference trajectories can help robot manipulators to travel through regions with higher manipulability and avoid kinematic singularities. To realize this goal, the manipulability measure is used as an important quantity in multiple techniques in this

article, including the constrained closed-loop inverse kinematics, the gradient descent of APF, and the biobjective cost function.

- 3) Similar to [21], a biobjective cost function is designed to simultaneously increase manipulability and reduce path length during path optimization. Nevertheless, we address the inverse kinematics problem caused by robot redundancy. By using the closed-loop inverse kinematics, a larger actual manipulability index can be assigned to each end-effector pose during the O-space path planning. In addition, this article considers the RRT* planner with the adaptive step size to improve the searching efficiency.

II. PRELIMINARIES

A. Kinematic Manipulability

Kinematic manipulability describes the capacity of a robot to change its position and orientation given a joint configuration. In control of a robot manipulator, the geometric Jacobian matrix is used to establish the velocity relationship between the C-space and the O-space. Let $J(\mathbf{q}) \in \mathbb{R}^{m \times k}$ denote the Jacobian matrix where m is the state dimension in O space and k is the degree of freedom in C-space. Then, we have

$$\dot{\mathbf{x}} = J(\mathbf{q})\dot{\mathbf{q}} \quad (1)$$

where $\dot{\mathbf{x}} \in \mathbb{R}^m$ is the end-effector velocity, and $\dot{\mathbf{q}} \in \mathbb{R}^k$ denotes the joint velocity. $\mathbf{q}$ denotes the joint position/angle. For redundant robot manipulators, the *manipulability measure* at the joint-space configuration $\mathbf{q}$ can be obtained from the Jacobian matrix as [12]

$$\omega(\mathbf{q}) = \sqrt{\det(J(\mathbf{q})J^T(\mathbf{q}))} \quad (2)$$

where $\det(*)$ denotes the determinant of an input matrix.

Remark 1: A robot at configurations $\mathbf{q}$ has a higher capability to move along at least one direction in its workspace if $\mathbf{q}$ gives higher manipulability measure in (2). On the other hand, the robot at configurations with lower manipulability measure has poor ability to move its end effector. Particularly, the robot loses the ability to move in one or more directions when it arrives at singular configurations (or *singularities*), where the manipulability measure is zero (or very close to zero).

B. Rapidly Exploring Random Tree

Based on the RRT method, the RRT* method adds two techniques to optimize the RRT method: 1) the RRT* stores the distance of each vertex relative to its parent vertex; 2) the RRT* method rewires an existing neighbor vertex to the newly added vertex that makes their cost decrease. To describe the procedure of the RRT* path planner, some definitions are given as follows.

- 1) $\mathcal{S}$: The 3-D work space of a robot.
- 2) $\mathcal{T} \in \mathcal{S}$: The RRT* tree that contains a set of vertices $\mathbf{X}$.
- 3) $\mathbf{x}_s \in \mathcal{S}$: Start position.
- 4) $\mathbf{x}_g \in \mathcal{S}$: Goal position.
- 5) $\mathbf{x}_r \in \mathcal{S}$: Random sampling state.
- 6) $\mathbf{x}_{\text{nearest}} \in \mathcal{T}$: The nearest existing vertex in the tree.
- 7) $\mathbf{x}_{\text{new}} \in \mathcal{T}$: The newly added vertex to the tree.

Algorithm 1: Basic RRT* Construction Algorithm.**Result:** Build an RRT* TreeInput: Initial position x_s , goal position x_g , vertex number K , and step size ϵ ;Output: RRT* tree $\mathcal{T}$;**for** $k = 1$ **to** K **do** $x_r \leftarrow \text{RandomState}()$; $x_{\text{nearest}} \leftarrow \text{NearestVertex}(x_r, \mathcal{T})$; $x_{\text{new}} \leftarrow \text{NewVertex}(x_r, x_{\text{nearest}}, \epsilon)$; $x_{\text{new}}.x_{\text{parent}} \leftarrow \text{FindParent}(x_{\text{new}}, \mathcal{T}, \mathcal{C}, R)$; $x_{\text{new}}.c_{\text{new}} \leftarrow \text{UpdateCost}(\mathcal{C}, x_{\text{new}}, x_{\text{parent}})$; $\mathcal{T}.\text{AddVertex}(x_{\text{new}}, x_{\text{nearest}})$; $\mathcal{T}.\text{AddEdge}(x_{\text{parent}}, x_{\text{new}})$; **return** $\mathcal{T}$;**end****Result:** Find a minimum-cost pathInput: Parent of the goal $x_{\text{parent}} = x_g.x_{\text{parent}}$, RRT* tree $\mathcal{T}$;Output: Optimal Path $\mathcal{P}_d$;**while** $x_{\text{parent}} \neq x_s$ **do** $x_{\text{parent}} = x_{\text{parent}}.x_{\text{parent}}$; $x_p = x_{\text{parent}}$; $\mathcal{P}_d.\text{AddWayPoint}(x_p)$; **return** $\mathcal{P}_d$;**end**

To find a path¹ using the RRT* method, the start (initial) position x_s and goal position x_g are given *a priori*. The number K of vertices that are added to the tree $\mathcal{T}$ and the RRT* step size $\epsilon > 0$ are also predetermined. The design of the RRT* path planner is described in Algorithm 1 [4]. The procedure can be summarized into two major steps.

1) **Constructing a Searching Tree $\mathcal{T}$:** The first step is to build an RRT* searching tree $\mathcal{T}$ through K iterations. To add a new vertex to the tree, the function “RandomState” grabs a random sampling state x_r from the search space $\mathcal{S}$. Then, the nearest vertex $x_{\text{nearest}} \in \mathcal{T}$ relative to x_r is found through the function “NearestVertex.” A step size ϵ is applied to manage distance between x_{nearest} and x_r . Specifically, if the distance between the random sample x_r and the nearest vertex x_{nearest} is smaller than the step size, i.e., $\|x_r - x_{\text{nearest}}\| \leq \epsilon$, the random sample is accepted as a new vertex and added to the tree $\mathcal{T}$, i.e., $x_n = x_r$. Otherwise, when $\|x_r - x_{\text{nearest}}\| > \epsilon$, along the direction between x_r and x_{nearest} , the function “NewVertex” selects a new state x_n such that $\|x_n - x_{\text{nearest}}\| = \epsilon$, as illustrated in Fig. 1.

Unlike the RRT method that directly adds an edge between x_{new} and x_{nearest} , the RRT* method evaluates the neighboring nodes $x_{\text{neighbor}} \in \mathcal{T}$, within a radius R of the new state x_{new} , i.e., $\|x_{\text{new}} - x_{\text{neighbor}}\| \leq R$. $R \geq \epsilon$. This step is referred to as “FindParent” in Algorithm 1 because the purpose is to find a parent node for x_{new} . A cost function $\mathcal{C}$ is designed for the evaluations. The neighbor that makes the minimum cost is selected as the parent node x_{parent} of the new state x_{new} .

¹Unless otherwise specified, in this article the term “path” refers to the end-effector’s translational trajectory.

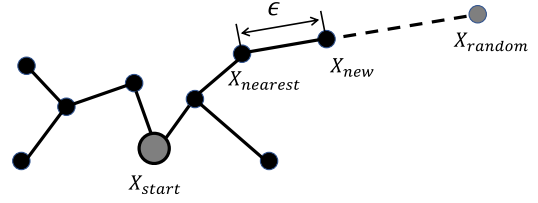


Fig. 1. Operation of incrementally constructing the RRT tree from samples drawn randomly from the search space.

This parent node is, then, wired to x_{new} , and their parent–child inheritance relationship is denoted by $x_{\text{new}}.x_{\text{parent}}$ in Algorithm 1. Accordingly, a new cost c_{new} that is evaluated by $\mathcal{C}$ is assigned to x_{new} through the function “UpdateCost.” x_{new} as a new vertex is, then, added into the tree $\mathcal{T}$. By running the RRT* sampling procedure for K iterations, the tree $\mathcal{T}$ branches and spreads into the unsearched space.

2) **Find a Minimum-Cost Path:** When the tree $\mathcal{T}$ has been constructed, the subsequent procedure, “Find a minimum-cost path,” is activated. A set of way points $x_w \in \mathbb{R}^p$ are selected to form a path $\mathcal{P}_d$ that connects the goal position x_g and the start position x_s . Starting from the goal state x_g , it finds the parent node of the goal $x_g.x_{\text{parent}}$. Denoting this parent node as x_{parent} and adding it as one of the waypoints that form the path. Then the parent node of the current x_{parent} is found and added to the waypoints. This procedure iterates until reaching the start position, i.e., $x_{\text{parent}} = x_s$. Then, a path with minimum cost has been found. Note that a path generated from the RRT* planner is usually required interpolation to gain a sufficient number of way points along the path $\mathcal{P}_d$.

III. PROBLEM STATEMENT

In this section, the RRT* path planning problems are formalized.

A. RRT* Step Size

In the RRT* method, the selection of the step size ϵ in Fig. 1 has a significant impact on the behavior of growing the RRT* tree. Specifically, by selecting a small step size ϵ , the RRT* planner can generate a smooth path with dense waypoints. However, as illustrated in Fig. 2(a), where a small step size is selected (i.e., $\epsilon = 1$), the tree fails to fill the space adequately. To fully explore the unsearched space, a higher vertex number K , and thus, a longer executing time is required. On the other hand, as shown in Fig. 2(d), setting a large step size $\epsilon = 15$ helps the tree branch out in searching the space and converge efficiently to the goal. However, the generated path may contain fewer waypoints and the distance between two sequential waypoints is relatively large. Such coarse paths may contain singularities that lead to poor robot performance in path-following control.

Theoretically, the issues caused by either small or large step size can be solved if the vertex number $K \rightarrow \infty$. However, in practice, it is realistic to give a finite vertex number K . Therefore, to smartly distribute the samples, we will focus on an adaptive strategy for determining the time-variant step size. Motivated

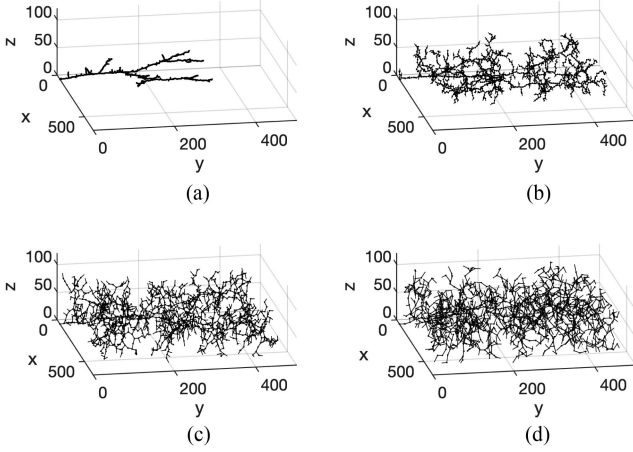


Fig. 2. Comparison of space-filling trees using the RRT* path planner (as in Algorithm 1) with different constant step sizes. (a) $\epsilon = 1$. (b) $\epsilon = 5$. (c) $\epsilon = 10$. (d) $\epsilon = 15$. In these examples, the search space size is $640 \times 480 \times 100$ and the sampling number is 2000.

by [18] and [20], where the adaptive step size is used to help avoid disadvantageous regions,² the adaptive step size problem is described as follows.

1) *Adaptive RRT* step size problem*: In the step of “NewVertex($\mathbf{x}_r$, $\mathbf{x}_{\text{nearest}}$, ϵ)” in Algorithm 1, when the nearest vertex $\mathbf{x}_{\text{nearest}}$ locates within disadvantageous regions, the adaptive step size $\epsilon(t)$ is activated to find the new vertex $\mathbf{x}_{\text{new}}$. Additionally, when $\mathbf{x}_{\text{nearest}}$ locates closer to the center of the disadvantageous regions $\mathcal{X}_O$, the smaller step size $\epsilon(t)$ is used. Otherwise, the step size $\epsilon(t)$ increases. When $\mathbf{x}_{\text{nearest}}$ locates outside the disadvantageous regions (i.e., $\mathbf{x}_{\text{nearest}} \notin \mathcal{X}_O$), the step size (ϵ_0) remains constant and $\epsilon_0 \geq \|\epsilon(t)\|$.

To facilitate the subsequent analysis of adaptive step size, a denotation (represented by E_s) that describes the searching efficiency is formulated by

$$E_s = \mu V_t / K \quad (3)$$

where V_t denotes the RRT* tree volume that is estimated by the convex hull. K is the vertex number as in Algorithm 1. μ is a scaling constant.

The searching efficiency is evaluated by the reciprocal of point density in the convex hull. In other words, to reach a certain volume of the RRT* tree, the searching efficiency is lower when it requires a higher number of vertices, and, thus, a longer time. Note that the searching efficiency may falsely be higher if the convex hull contains only a few vertices. This case can be easily avoided by presetting sufficient vertex number K to increase the iterations in Algorithm 1.

B. RRT* Cost Function

In path planning for redundant robot manipulators, merely finding the shortest path may give rise to singularities that significantly deteriorate the performance of an industrial robot

²Disadvantageous regions refer to the space that contains factors that can lead to poor performance of the robot. In this article, the disadvantageous region refers to space that exists singularities measured by (2).

arm. In this article, a biobjective cost function is to be designed as a function of manipulability and path length such that the paths generated from the RRT* planner are singularity-free and short (although not necessarily the shortest). The problem is, then, defined as follows.

1) *Bi-objective cost function*: In the steps of “FindParent” and “UpdateCost” in Algorithm 1, the cost function $\mathcal{C}$ for evaluating the new node $\mathbf{x}_{\text{new}}$ is designed by the summation of both manipulability measure and incremental step length, accumulating from the start point. Then, a set of way points $[\mathbf{x}_w(1), \mathbf{x}_w(2), \dots, \mathbf{x}_w(p)]$ are found to form an optimal (i.e., minimum cost) path $\mathcal{P}_d$. Let c_w denote the cost of the way points, and the path $\mathcal{P}_d$ satisfies the following condition:

$$\begin{aligned} \mathcal{P}_d &= [\mathbf{x}_w(1), \mathbf{x}_w(2), \dots, \mathbf{x}_w(p)] \in \mathbb{R}^p \\ \text{s.t. } \min &\left(\sum_{i=1}^{i=p} c_w(i) \right). \end{aligned} \quad (4)$$

Another issue to be addressed is the use of manipulability in O-space. For redundant manipulators, the mapping from the O-space and the C-space is not unique. Therefore, using inverse kinematics may lead to more than one solution or even infinite solutions. As a result, manipulability directly obtained from the joint data is not unique for a given O-space position, which motives us to consider the following problem.

2) *Mapping between O-space and C-space*: For redundant serial robot manipulators, once the new vertex $\mathbf{x}_{\text{new}}$ in O-space is determined in the step of “NewVertex” in Algorithm 1, effort seeks to select one appropriate joint state $\mathbf{q}_{\text{new}}$ from all possible solutions so that the manipulability measure w_{new} assigned to $\mathbf{x}_{\text{new}}$ is unique and facilitates the optimal solution in (4).

IV. MAIN RESULTS

In this section, the main theoretical results will be presented, including the biobjective cost function, the constrained closed-loop inverse kinematics, and the adaptive step size.

A. Biobjective Cost Function

To consider the path length as a cost in the RRT* scheme, a cost component $\mathcal{C}_{\text{dist}}(\mathbf{x}_{\text{new}})$ can be given as follows:

$$\begin{aligned} \mathcal{C}_{\text{dist}}(\mathbf{x}_{\text{new}}) &= d(\mathbf{x}_{\text{new}} \cdot \mathbf{x}_{\text{parent}}, \mathbf{x}_{\text{new}}) \\ &+ \sum_{i=1}^{i=n} d(\mathbf{x}_{\text{parent}}(i) \cdot \mathbf{x}_{\text{parent}}, \mathbf{x}_{\text{parent}}(i)) \end{aligned} \quad (5)$$

where n is the number of parent nodes that it takes sequentially from the start point $\mathbf{x}_s$ to the newly added vertex $\mathbf{x}_{\text{new}}$. Therefore, in (5) we have $\mathbf{x}_{\text{new}} \cdot \mathbf{x}_{\text{parent}} = \mathbf{x}_{\text{parent}}(n)$ and $\mathbf{x}_{\text{parent}}(1) \cdot \mathbf{x}_{\text{parent}} = \mathbf{x}_s$. The function $d(\cdot)$ returns the Euclidean distance of its inputs.

The function $\mathcal{C}_{\text{dist}}(\mathbf{x}_{\text{new}})$ calculates the accumulative cost of the Euclidean distance from the initial position $\mathbf{x}_s$ to the newly added vertex $\mathbf{x}_{\text{new}}$. To take into account the manipulability, a cost component $\mathcal{C}_M(\mathbf{x}_{\text{new}})$ that evaluates the manipulability cost at

the position $\mathbf{x}_{\text{new}}$ with the configuration $\mathbf{q}_{\text{new}}$ is given as

$$\mathcal{C}_M(\mathbf{x}_{\text{new}}) = \frac{1}{n+1} \left[\frac{1}{\omega(\mathbf{q}_{\text{new}})} + \sum_{i=1}^n \frac{1}{\omega(\mathbf{q}_{\text{parent}}(i))} \right] \quad (6)$$

where $\mathbf{q}_{\text{parent}}(i)$ is the corresponding joint state of the parent node $\mathbf{x}_{\text{parent}}(i)$. The manipulability measure $\omega(\mathbf{q}_{\text{new}})$ and $\omega(\mathbf{q}_{\text{parent}}(i))$ are calculated by (2).

Then, the solution to find a minimum-cost path, as described in (4), is given in Theorem 1, where the cost c_w is given by a biobjective cost function $\mathcal{C}(\mathbf{x}_{\text{new}})$.

Theorem 1: In the RRT* planner, the cost of each newly added vertex $\mathbf{x}_{\text{new}}$ is evaluated by (5) and (6), and a minimum-cost path can be almost surely found with a biobjective cost function given by

$$\mathcal{C}(\mathbf{x}_{\text{new}}) = \min_{\mathbf{x}_{\text{new}}} \|\beta \mathcal{C}_{\text{dist}}(\mathbf{x}_{\text{new}}) + (1 - \beta) \mathcal{C}_M(\mathbf{x}_{\text{new}})\| \quad (7)$$

where the weight factor β sets a tradeoff between $\mathcal{C}_{\text{dist}}(\mathbf{x}_{\text{new}})$ and $\mathcal{C}_M(\mathbf{x}_{\text{new}})$. $0 \leq \beta \leq 1$.

Proof: According to the framework in [8] and [23], the cost of the minimum-cost path converges to the minimal cost almost surely, if the following three conditions of the cost function are satisfied.

Condition 1. Additivity: The cost function must be additive. This can be easily proved by (5)–(7), where the cost functions are defined in an additive fashion.

Condition 2. Monotonicity: Let $\sigma_1 | \sigma_2$ denote the concatenation of path segments σ_1 and σ_2 . Then, the cost function is monotonic if $\Delta \mathcal{C}(\sigma_1) \leq \Delta \mathcal{C}(\sigma_1 | \sigma_2)$. In (5) and (6), the incremental costs are calculated from the Euclidean distance and manipulability measure, respectively. Therefore, the incremental costs are both nonnegative, i.e., $\Delta \mathcal{C}_{\text{dist}}(*) \geq 0$ and $\Delta \mathcal{C}_M(*) \geq 0$, implying monotonicity.

Condition 3. Lipschitz Continuity: In (5), the cost function is directly given by the incremental path length. Therefore, we have $\Delta \mathcal{C}_{\text{dist}}(\sigma_1) = \sigma_1$ and $\Delta \mathcal{C}_{\text{dist}}(\sigma_2) = \sigma_2$, which satisfy the Lipschitz condition of $|\Delta \mathcal{C}_{\text{dist}}(\sigma_1) - \Delta \mathcal{C}_{\text{dist}}(\sigma_2)| \leq K_{Lz} |\sigma_1 - \sigma_2|$ with the Lipschitz constant K_{Lz} equal to 1. Therefore, the cost function in (5) is 1-Lipschitz continuous. In (6), let ω_1 and ω_2 be the equivalent inputs of the path segments σ_1 and σ_2 . Therefore, we have $\Delta \mathcal{C}_M(\sigma_1) = \frac{1}{(n+1)\omega_1}$ and $\Delta \mathcal{C}_M(\sigma_2) = \frac{1}{(n+1)\omega_2}$, which gives

$$\begin{aligned} |\Delta \mathcal{C}_M(\sigma_1) - \Delta \mathcal{C}_M(\sigma_2)| &= \left| \frac{1}{(n+1)\omega_1} - \frac{1}{(n+1)\omega_2} \right| \\ &= \frac{1}{(n+1)|\omega_1\omega_2|} |\omega_1 - \omega_2|. \end{aligned}$$

Let the Lipschitz constant $K_{Lz} = \frac{1}{(n+1)|\omega_1\omega_2|}$. As $\frac{1}{(n+1)|\omega_1\omega_2|} \geq 0$, the cost function in (6) is Lipschitz continuous. Thus, the cost function that consists of the cost in (5) and (6) is also Lipschitz continuous. As the cost function in (7) satisfies the above three conditions, the cost of the minimum-cost path converges to the minimal cost almost surely. This completes the proof. ■

In reality, the Cartesian distance and the manipulability measure may be on drastically different scales. Therefore, they need to be rescaled so that each feature is equally important in the

cost function in (7). In this work, the rescaling is implemented by the min-max normalization formulated as

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (8)$$

such that both the Cartesian distance and the manipulability measure are rescaled into the range of $[0, 1]$.

B. Constrained Closed-Loop Inverse Kinematics

In (6), given an end-effector position, it requires to calculate the joint states and, then, the manipulability cost. For nonredundant robot manipulators, the kinematic mapping can be established using the knowledge of the manipulator Jacobian matrix and its inverse (in this work we use the pseudo-inverse). Let $J^+(\mathbf{q}) \in \mathbb{R}^{k \times m}$ denote the pseudo-inverse of the matrix $J(\mathbf{q})$, and $J^+(\mathbf{q})\dot{\mathbf{x}}$ provides a least-square solution with minimum norm and satisfies the Moore–Penrose condition

$$\min_{\dot{\mathbf{q}}} \|\dot{\mathbf{q}}\| \quad \text{s.t.} \quad \min_{\dot{\mathbf{q}}} \|\dot{\mathbf{x}} - J(\mathbf{q})\dot{\mathbf{q}}\|. \quad (9)$$

Then, the joint positions $\mathbf{q} \in \mathbb{R}^k$ can be computed by integrating the velocity solution over time with known initial conditions. However, for kinematically redundant³ manipulators, the condition in (9) cannot be satisfied due to the redundancy. To address this issue, we adopt the constrained closed-loop inverse kinematics [24] that is able to deal with the redundant degrees of freedom. The constrained closed-loop inverse kinematics that establish the velocity mapping between the C-space and then O-space is described as follows.

Lemma 1 ([24]): To handle the redundant degrees of freedom, an arbitrary joint velocity vector $\dot{\mathbf{q}}_0 \in \mathbb{R}^k$ that produces no motion at the end-effector is used. The vector $\dot{\mathbf{q}}_0$ is projected onto a nonempty null space $N(J)$. The general closed-loop inverse solution can be written as

$$\dot{\mathbf{q}} = J^+(\mathbf{q})\kappa_c \mathbf{e} + (I - J^+(\mathbf{q})J(\mathbf{q}))\dot{\mathbf{q}}_0 \quad (10)$$

where $\mathbf{e} = \mathbf{x} - \mathbf{x}_d \in \mathbb{R}^m$ denotes the closed-loop feedback. $\mathbf{x}_d$ and $\mathbf{x}$ are the reference and the actual constraint end-effector position vectors. κ_c is a tunable gain. The matrix $(I - J^+(\mathbf{q})J(\mathbf{q}))$ is a homogeneous projector of the joint vector $\dot{\mathbf{q}}_0$ onto $N(J)$.

The analysis and proof of Lemma 1 have been detailed in [24], and, therefore, are omitted here. The solution to (10) can be found by iterating the closed-loop until a reaching condition has been satisfied, such as reaching the minimal error $\mathbf{e}_m$ such that $\mathbf{e} \leq \mathbf{e}_m$ or predetermined iterations. Note that the computation efficiency may be reduced by the iterative process. A typical choice of the null-space velocity vector is

$$\dot{\mathbf{q}}_0 = \alpha \left(\frac{\partial \mathcal{H}(\mathbf{q})}{\partial \mathbf{q}} \right)^T \in \mathbb{R}^k \quad (11)$$

where $\mathcal{H}(\mathbf{q})$ represents a scalar objective function of the joint states. In this work, we aim at exploiting the redundancy to avoid violating the joint limits and to seek a feasible configuration with

³A robot manipulator is said to be redundant when it can reach a specified position with more than one configuration of the linkages.

higher manipulability. Therefore, the objective function can be rewritten as

$$\mathcal{H}(\mathbf{q}) = \mathcal{H}_l(\mathbf{q}) + \mathcal{H}_M(\mathbf{q}). \quad (12)$$

The term of $\mathcal{H}_l(\mathbf{q})$ denotes the objective in terms of joint limits, and is given by

$$\mathcal{H}_l(\mathbf{q}) = -\frac{1}{2k} \sum_{i=1}^k \left(\frac{\mathbf{q}(i) - \bar{\mathbf{q}}(i)}{\mathbf{q}_M(i) - \mathbf{q}_m(i)} \right)^2 \quad (13)$$

where $\mathbf{q}_M \in \mathbb{R}^k$ and $\mathbf{q}_m \in \mathbb{R}^k$ represent the upper limit and lower limit of joint states, respectively. $\bar{\mathbf{q}} \in \mathbb{R}^k$ denotes the middle of the joint range. k denotes the DOF as in (1). Then we can obtain the gradient of $\mathcal{H}_l(\mathbf{q})$ as

$$\frac{\partial \mathcal{H}_l(\mathbf{q})}{\partial \mathbf{q}} = ((\mathbf{q} - \mathbf{q}_m)^{\circ 2} \circ (2\mathbf{q} - \mathbf{q}_M - \mathbf{q}_m)) \oslash (4(\mathbf{q}_M - \mathbf{q})^{\circ 2} \circ (\mathbf{q} - \mathbf{q}_m)^{\circ 2}) \in \mathbb{R}^k \quad (14)$$

where $\circ$ denotes the Hadamard product, $(*)^{\circ 2}$ the Hadamard power, and $\oslash$ the Hadamard division.

As to the manipulability measure, simply let $\mathcal{H}_M(\mathbf{q}) = \omega_i$ as in (2). As we only consider the translational motion in this article, the translational components J_v of the Jacobian Matrix J is used. Then, the gradient of $\mathcal{H}_M(\mathbf{q})$ can be expressed as

$$\begin{aligned} \frac{\partial \mathcal{H}_M(\mathbf{q})}{\partial \mathbf{q}} &= -\frac{1}{2\mathcal{H}_M} \frac{\partial \det(J_v(\mathbf{q})J_v^T(\mathbf{q}))}{\partial \mathbf{q}} \\ &= \begin{bmatrix} \mathcal{H}_M \text{vec}(J_v(\mathbf{q})\mathbf{H}_t^T(1))^T \text{vec}(J_v(\mathbf{q})J_v^T(\mathbf{q})) \\ \mathcal{H}_M \text{vec}(J_v(\mathbf{q})\mathbf{H}_t^T(3))^T \text{vec}(J_v(\mathbf{q})J_v^T(\mathbf{q})) \\ \vdots \\ \mathcal{H}_M \text{vec}(J_v(\mathbf{q})\mathbf{H}_t^T(k))^T \text{vec}(J_v(\mathbf{q})J_v^T(\mathbf{q})) \end{bmatrix}^T \in \mathbb{R}^k \end{aligned} \quad (15)$$

where $\text{vec}(\cdot)$ denotes the vectorization operation on the input matrix. $\mathbf{H}_t \in \mathbb{R}^{\frac{m}{2} \times k \times k}$ represents the translational manipulator Hessian, and can be calculated using the full pose Jacobian matrix [25], [26]. $\mathbf{H}_t = [\mathbf{H}_t\{1\}, \mathbf{H}_t\{2\}, \dots, \mathbf{H}_t\{k\}]$ and $\mathbf{H}_t\{i\} = [\mathbf{H}_t\{i\}(1), \mathbf{H}_t\{i\}(2), \dots, \mathbf{H}_t\{i\}(k)] \in \mathbb{R}^{\frac{m}{2} \times k}$. The component $\mathbf{H}_t\{i\}(j) \in \mathbb{R}^{\frac{m}{2} \times 1}$ ($i, j = 1, 2, \dots, k$) is given by

$$\mathbf{H}_t\{i\}(j) = J_{\omega,a}(\mathbf{q}) \times J_{v,b}(\mathbf{q}) \in \mathbb{R}^{\frac{m}{2} \times 1} \quad (16)$$

where $\times$ denotes the cross product. $a = \min([i, j])$ and $b = \max([i, j])$. The translational and orientational Jacobian components, denoted by J_v and J_ω , respectively, are indicated as follows (e.g., $m = 6$):

$$J = \begin{bmatrix} \underbrace{J_{v,1}}_{J_{11}} & \underbrace{J_{v,2}}_{J_{12}} & \cdots & \underbrace{J_{v,k}}_{J_{1k}} \\ J_{21} & J_{22} & \cdots & J_{2k} \\ J_{31} & J_{32} & \cdots & J_{3k} \\ \vdots & \vdots & \ddots & \vdots \\ J_{41} & J_{42} & \cdots & J_{4k} \\ J_{51} & J_{52} & \cdots & J_{5k} \\ J_{61} & J_{62} & \cdots & J_{6k} \end{bmatrix} \begin{Bmatrix} J_v \\ J_\omega \end{Bmatrix}$$

By using the solutions in (10), (11), (14), and (15), the kinematic redundancy can be exploited to keep the manipulator

off joint limits and helps the RRT* planner find a path with higher manipulability.

C. Adaptive RRT* Step Size

As suggested in [20], exploiting manipulability-biased node selection criterion can help improve the sampling efficiency. This motivates the design of manipulability-based adaptive step size in this work. Specifically, in the disadvantageous regions with low manipulability, the searching step size reduces to carefully avoid the singularities. In contrast, in regions with higher manipulability, a larger step size is used to speed up the space exploring. In addition, we wish the RRT* tree to quickly converge to the goal region. Thus, the step size becomes smaller in the goal region. Thus, an APF gradient descent strategy is utilized to adaptively change the step size ϵ . As we consider the disadvantageous regions in this article, the repulsive potential is used. To choose the new vertex $\mathbf{x}_{\text{new}}$ when the node $\mathbf{x}_{\text{nearest}}$ is determined (see Algorithm 1), the repulsive potential is given by

$$U(\mathbf{x}_{\text{nearest}}) = U_\omega(\mathbf{x}_{\text{nearest}}) + U_{\text{dist}}(\mathbf{x}_{\text{nearest}}) \quad (17)$$

where $U(\mathbf{x}_{\text{nearest}})$ denotes the total repulsive potential at the vertex $\mathbf{x}_{\text{nearest}}$. $U_\omega(\mathbf{x}_{\text{nearest}})$ denotes the potential function of the manipulability measure and $U_{\text{dist}}(\mathbf{x}_{\text{nearest}})$ represents the potential function of the Euclidean distance from the nearest vertex $\mathbf{x}_{\text{nearest}}$ to the goal $\mathbf{x}_g$. In what follows, the augment of $\mathbf{x}_{\text{nearest}}$ is omitted unless otherwise denoted. Then, the potential function U_ω is given as

$$U_\omega = \begin{cases} \frac{1}{2}\eta_\omega \left(\frac{1}{\omega(\mathbf{q}_{\text{nearest}})} - \frac{1}{\omega^*} \right)^2, & \omega(\mathbf{q}_{\text{nearest}}) \leq \omega^* \\ 0, & \omega(\mathbf{q}_{\text{nearest}}) > \omega^* \end{cases} \quad (18)$$

where $\eta_\omega > 0$ is a constant that determines the strength of the repulsive field. The influence range in terms of manipulability measure is denoted by ω^* . Similarly, another potential component U_{dist} is given as

$$U_{\text{dist}} = \begin{cases} \frac{1}{2}\eta_{\text{dist}} \left(\frac{1}{d(\mathbf{x}_{\text{nearest}})} - \frac{1}{d^*} \right)^2, & d(\mathbf{x}_{\text{nearest}}) \leq d^* \\ 0, & d(\mathbf{x}_{\text{nearest}}) > d^* \end{cases} \quad (19)$$

where $d(\mathbf{x}_{\text{nearest}})$ represents the Euclidean distance from $\mathbf{x}_{\text{nearest}}$ to the goal $\mathbf{x}_g$. η_{dist} is a positive constant that determines the strength of the repulsive field. The influence range in terms of the Euclidean distance is denoted by d^* .

The gradient of the repulsive potential in (17) is given with respect to the configuration $\mathbf{q}_{\text{nearest}}$, and we have

$$\nabla U = \nabla U_\omega + \nabla U_{\text{dist}}. \quad (20)$$

Then, the potential gradient components, $\nabla U_\omega \in \mathbb{R}^k$ and $\nabla U_{\text{dist}} \in \mathbb{R}^k$ are given as follows:

$$\nabla U_\omega = \begin{cases} \eta_\omega \left(\frac{1}{\omega(\mathbf{q}_{\text{nearest}})} - \frac{1}{\omega^*} \right) \frac{1}{\omega^2(\mathbf{q}_{\text{nearest}})} \frac{\partial \omega(\mathbf{q}_{\text{nearest}})}{\partial \mathbf{q}_{\text{nearest}}}, & \omega(\mathbf{q}_{\text{nearest}}) \leq \omega^* \\ 0_{k \times 1}, & \omega(\mathbf{q}_{\text{nearest}}) > \omega^* \end{cases}$$

where the gradient $\frac{\partial \omega(\mathbf{q}_{\text{nearest}})}{\partial \mathbf{q}_{\text{nearest}}}$ can be calculated by (15). The potential gradient component $\nabla U_{\text{dist}} \in \mathbb{R}^k$ is given as

$$\nabla U_{\text{dist}} = \begin{cases} \eta_{\text{dist}} \left(\frac{1}{d(\mathbf{x}_{\text{nearest}})} - \frac{1}{d^*} \right) \frac{1}{d^2(\mathbf{x}_{\text{nearest}})} \frac{\partial d(\mathbf{x}_{\text{nearest}})}{\partial \mathbf{q}_{\text{nearest}}} & d(\mathbf{x}_{\text{nearest}}) \leq d^* \\ 0, & d(\mathbf{x}_{\text{nearest}}) > d^*. \end{cases}$$

As the Euclidean distance is given in the form of, for example, $d = \sqrt{(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2}$, we have

$$\begin{aligned} \frac{\partial d}{\partial \mathbf{q}} &= \frac{1}{2} [(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2]^{-\frac{1}{2}} \\ &\quad \left[2(x - x_0) \frac{\partial x}{\partial \mathbf{q}} + 2(y - y_0) \frac{\partial y}{\partial \mathbf{q}} + 2(z - z_0) \frac{\partial z}{\partial \mathbf{q}} \right] \\ &= [(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2]^{-\frac{1}{2}} \\ &\quad [x - x_0, y - y_0, z - z_0] \left[\frac{\partial x}{\partial \mathbf{q}}, \frac{\partial y}{\partial \mathbf{q}}, \frac{\partial z}{\partial \mathbf{q}} \right]^T \\ &= \frac{\vec{d} J_v(\mathbf{q})}{d} \end{aligned} \quad (21)$$

where $\vec{d} = [x - x_0, y - y_0, z - z_0]$, and $J_v(\mathbf{q})$ is the translational Jacobian matrix as in (15). $\frac{\partial d}{\partial \mathbf{q}} \in \mathbb{R}^k$. Therefore, we obtain

$$\frac{\partial d(\mathbf{x}_{\text{nearest}})}{\partial \mathbf{q}_{\text{nearest}}} = \frac{\vec{d}(\mathbf{x}_{\text{nearest}}) J_v(\mathbf{q}_{\text{nearest}})}{d(\mathbf{x}_{\text{nearest}})} \in \mathbb{R}^k.$$

With the definitions in (17) and (20), the following theorem is immediate.

Theorem 2: The step size is determined by the gradient of the artificial potential as

$$\epsilon = \epsilon_0 - \gamma \|\nabla U(\mathbf{x}_{\text{nearest}})\| \quad (22)$$

such that the step size is automatically reduced when it comes to regions with lower manipulability and/or close to the goal. ϵ_0 is the constant step size that is used when $\mathbf{x}_{\text{nearest}}$ is outside the range of potential fields. $\gamma > 0$ provides additional flexibility to tune the overall potential strength.

Remark 2: At this stage, values of the potential strength and the influence range (i.e., η_ω and ω^* in (18) and η_{dist} and d^* (19)) are chosen at the designer's discretion and the individual robots' application-specific performance requirements. For example, the potential strength η_ω in (18) can be set to a higher value in applications where the adaptive step size is required to be sensitive to the change of manipulability measure.

D. Proportional-Derivative (PD) Controller

For the purpose of numerical simulations. A PD controller is designed to drive the robot toward the trajectories generated from the RRT* planner. For redundancy resolution, a velocity-based control method with joint velocity integration [27] is used. Given the desired velocity $\dot{\mathbf{x}}_d$ and position $\mathbf{x}_d$ of the robot end-effector, a reference velocity command $\dot{\mathbf{q}}_r \in \mathbb{R}^k$ in C-space is given as

$$\dot{\mathbf{q}}_r = J^+(\dot{\mathbf{x}}_d + \lambda_p(\mathbf{x}_d - \mathbf{x})) - (I - J^+(\mathbf{q})J(\mathbf{q}))\dot{\mathbf{q}}_0 \quad (23)$$

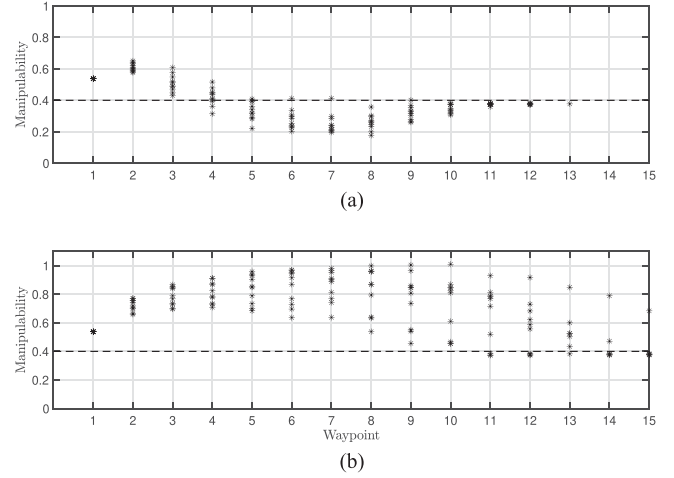


Fig. 3. Manipulability measure of the generated waypoints using the RRT* planner. (a) Case 1: Without the manipulability cost function. (b) Case 2: With the manipulability cost function.

where gain λ_p is positive and determined by tuning. I denotes an identity matrix. The vector $\dot{\mathbf{q}}_0$ is the null-space velocity vector, as in (11). Then, the control signal $\boldsymbol{\tau}$ is designed as

$$\boldsymbol{\tau} = K_p(\mathbf{q} - \mathbf{q}_r) + K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_r) \quad (24)$$

where $K_p \in \mathbb{R}^{k \times k}$ and $K_d \in \mathbb{R}^{k \times k}$ are control gains.

V. SIMULATION RESULTS

In this section, the numerical analysis on benefits of using the manipulability-based cost function and the adaptive step size in the RRT* path planning method has been carried out. In the numerical simulations, we used a six degrees-of-freedom (6-DOF) ($k = 6$) FANUC-M-20iA industrial robot represented by a Newton-Euler equations model [28]. In the PD controller, the control gains were selected as $K_p = \text{diag}([3000, 12000, 10000, 700, 35, 0.1])$, and $K_d = \text{diag}([200, 800, 700, 80, 10, 0.1])$. Unless otherwise specified, the vertex number were set as $K = 20000$ in the RRT* path finding strategy. In (3), $\mu = 10000$. To test the benefits of using the cost function in (7) in the RRT* planner to find a path that contains waypoints with higher manipulability, two special cases were performed for comparison as follows: *Case 1:* RRT* path planning with $\beta = 1$ and *Case 2:* RRT* path planning with $\beta = 0$. To obtain a fair comparison, the path planning procedures in both Case 1 and Case 2 were repeated ten times. Each path contains about 15 waypoints, as shown in Fig. 3. Thus, at each waypoint index, there are ten different manipulability values denoted by black dots.

In Case 2, the cost function of manipulability measure [i.e., (6)] is used as $\beta = 0$. Therefore, the RRT* path planner seeks to find an optimal path with higher manipulability, which has been verified by the results in Fig. 3(b). It shows that the manipulability values of most of the waypoints are over 0.4 (in this simulation, the highest manipulability index is 1.043). By contrast, in Case 1, 60% of the waypoints have the manipulability lower than 0.4, especially for the waypoints that are closer to

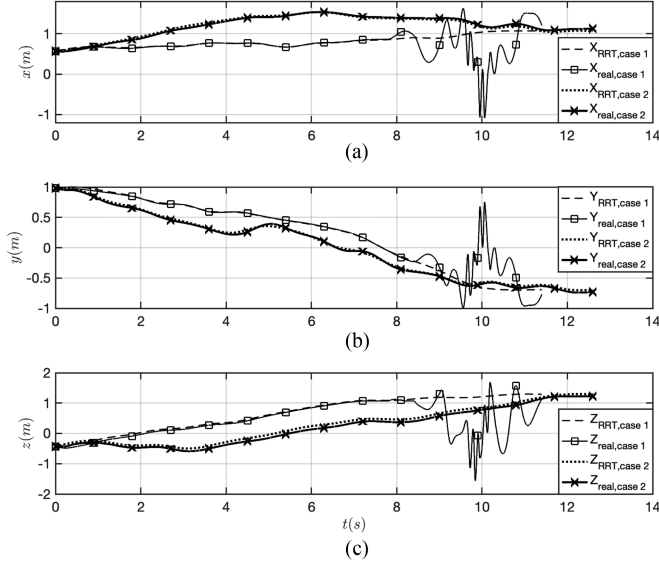


Fig. 4. Results of the robot end-effector tracking the paths generated from the RRT* planners in Case 1 and Case 2 along x , y , and z direction.

the goal (i.e., way points indexed 5 ~ 13), as shown in Fig. 3(a). The comparisons in Fig. 3 prove the good ability of the proposed RRT* method in finding paths with high manipulability.

Note that the reference manipulability index 0.4 in Fig. 3 does not have a particular physical meaning. It is merely used to facilitate the comparison of the manipulability distribution between Case 1 and Case 2. Therefore, to further demonstrate the benefits of Case 2, the trajectories generated in Case 1 and Case 2, respectively, were used for the path following control [i.e., the PD controller, as in (24)]. The results of the end-effector trajectory tracking are depicted in Fig. 4. It can be seen that without considering the manipulability in the path planning strategy (i.e., Case 1), the robot can present bad performance in following the generated path. For example, along the x direction in Case 1, the actual robot path $X_{\text{real, case 1}}$ fails to track the planned path $X_{\text{RRT, case 1}}$ after 8 s. This is caused by the sudden and large change in the joint motion when implementing the inverse kinematics, although the planned end-effector trajectories turn out smooth. In contrast, in Case 2 where the RRT* planner finds an optimal path with higher manipulability, the robot can smoothly track the desired trajectories (e.g., the actually robot end effector $X_{\text{real, case 2}}$ well tracks the generated path $X_{\text{RRT, case 2}}$). Because the RRT* path planner in Case 2 searches for waypoints that are with maximum manipulability in the neighborhood, the manipulability of the trajectory in Case 2 tends to change smoothly, which, in other words, avoids the sudden change in the joint motions. Therefore, the comparisons in Figs. 3 and 4 show that the trajectory tracking in Case 2 outperforms that in Case 1.

Case 1 and Case 2 demonstrate two special cases where the weight gain $\beta = 0$ and $\beta = 1$. In (7), the weight gain β can be adjusted in $[0, 1]$ to trade off the costs of manipulability and path length. Therefore, the effect of different β values was examined and the results are demonstrated in Fig. 5. Fig. 5(a) compares the manipulability profiles of the end-effector trajectories generated

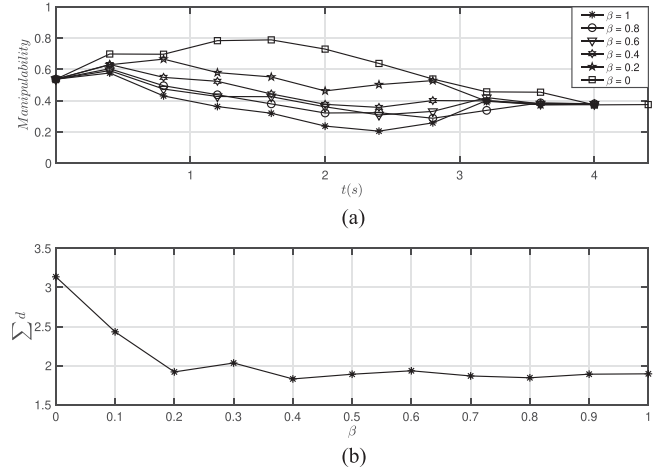


Fig. 5. (a) Manipulability profile of the generated path (contains 12 way points in this example) with different gains β . (b) The path length $\sum d$ changing over the gain β .

with different β values. For example, when $\beta = 0$, the cost function only consists of the manipulability cost and, therefore, a path (with way points denoted by square $\square$) with higher manipulability is found. Then, as β increases, the importance of the manipulability cost decreases in (7), and, therefore, the manipulability of the generated paths is reduced. For example, the manipulability of the path is low when $\beta = 1$ as can be seen from the path with way points denoted by asterisk $*$. The results in Case 1 and Case 2 (e.g., Fig. 3) are also in line with those in Fig. 5. On the other hand, Fig. 5(b) demonstrates the relationship between the path length and the gain β . It shows that when $\beta < 0.2$, the generated path length increases drastically, while the path length changes less significantly (about 2 m) for $\beta \geq 0.2$. Therefore, when using the proposed RRT* path planning method, we suggest that β is selected no less than 0.2 if generating shorter paths is under consideration.

Another important design of the proposed RRT* method is the adaptive step size. In the simulation testing, the constant step size ϵ_0 [as in (22)] was selected as the 3% of side length of the work space. To use the potential field in (18) and (19), the influence ranges were set as $\omega^* = 0.5$ and $d^* = 0.01$ and the gains to adjust the potential field strength were given as $\eta_\omega = 0.02$ and $\eta_{\text{dist}} = 0.005$. As stated in Remark 2, the influence ranges and the potential field strength in (18) and (19) can be carefully tuned to acquire a appropriate change of $\beta \|\nabla U(\mathbf{x}_{\text{nearest}})\|$ in (22). Denoting $\Delta_\epsilon = \beta \|\nabla U(\mathbf{x}_{\text{nearest}})\|$, an example distribution of Δ_ϵ is presented in Fig. 6, which can visually facilitate the tuning. As shown in Fig. 6, the sample distribution of the Δ_ϵ value is nonuniform, ranging in $[0, 0.013]$ m. In other words, the step size ϵ in (22) varies in $[0.007, 0.02]$ m as $\epsilon_0 = 0.02$ m.

According to the potential gradient in (20), $\Delta_\epsilon = 0$ when the robot locates in the higher manipulability regions (i.e., $\omega(\mathbf{q}) > \omega^*$) and away from the goal (i.e., $d(\mathbf{x}) > d^*$). In contrast, $0 < \Delta_\epsilon \leq 0.013$ m when the robot locates either in the lower manipulability regions (i.e., $\omega(\mathbf{q}) \leq \omega^*$) or close to the goal (i.e., $d(\mathbf{x}) \leq d^*$). Then, the step size changes adaptively based on (22). To demonstrate the benefits of using the adaptive

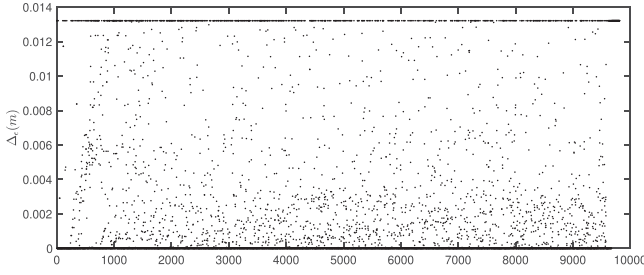


Fig. 6. Distribution of step size variance $\Delta\epsilon$.

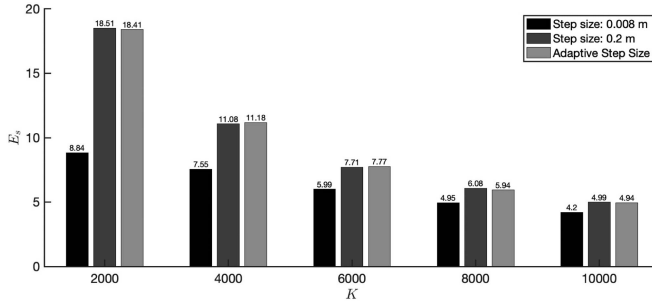


Fig. 7. Searching efficiency E_s of three cases where the step size are given as a small constant (0.008 m), a large constant (0.2 m), and adaptive step size, accordingly. The three cases were run with different vertex number K from 2000 to 10 000.

step size, searching efficiency [as defined in (3)] of three different step sizes has been examined, including a large constant step size 0.02 m, a small constant step size 0.008 m, and the proposed adaptive step size. The results of the searching efficiency with the three different step sizes are shown in Fig. 7. According to the demonstrations in Fig. 2, with a larger step size, the RRT* strategy searches the space more efficiently, which, again, can be seen from the comparisons between the two constant step sizes: 0.008 and 0.02 m in Fig. 7. For example, given $K = 2000$ vertex number, the efficiency is 18.51 with the step size 0.02 m, which is almost twice as efficient as that with the step size 0.008 m, where the efficiency is only 8.84. Fig. 7 also demonstrates that the RRT* method can run in high efficiency with the proposed adaptive step size. For example, given 2000 nodes, the efficiency is 18.41 with the adaptive step size, which is approximately as high as that with the large constant step size 0.02 m. The searching efficiency was also tested with different vertex numbers: $K = 2000$ to $K = 10000$. Fig. 7 shows that using the proposed adaptive step size or a larger constant step size still allows the higher efficiency, but the efficiency difference becomes smaller between the three types of step size as the different vertex number increases (e.g., $K = 10000$). The reason is that in applications with robot manipulators, the work space is limited. As in (3), significantly increasing the vertex number K can result in the similar searching efficiency although different step sizes are used. Therefore, the ability of automatic step size adjustment using the adaptive step size is also examined to present the benefit of using the adaptive RRT* method. Fig. 8 presents the results of three examples where RRT* method with the adaptive step size searches in different subworkspaces

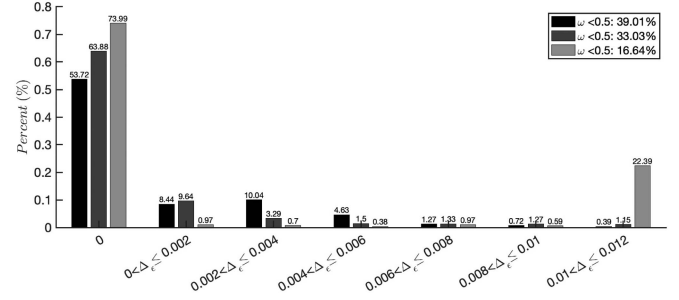


Fig. 8. Distribution (in percentage) of the step size variance in three scenarios where the percentages of the manipulability lower than 0.5 in the workspace are different, that is, 39.01%, 33.03%, and 16.64%.

TABLE I
COMPARISON WITH OTHER RRT-BASED METHODS

Methods	Manipulability average (minimum)	Efficiency
Adaptive RRT [18]	0.689(0.454)	3.124
Manipulability-based RRT* [21]	0.933(0.802)	1.927
Our method	1.094(0.979)	3.127

that have different percentages of manipulability lower than 0.5 (denoted as $\omega \leq 0.5$ in the remaining article). The purpose of the three examples is to examine if the step size can change more (less) frequently as the percentage of $\omega \leq 0.5$ increases (decreases) in the searching area. It can be seen in Fig. 8 that as the percentage of $\omega \leq 0.5$ increases from 16.64% to 39.01%, the percentage of nonzero $\Delta\epsilon$ also increase from 26.01% to 46.28% so that the finer step sizes ϵ in (22) are used when searching in the area with lower manipulability. Fig. 8 also summarizes the distribution of the nonzero $\Delta\epsilon$ values. The percentages of different $\Delta\epsilon$ values reflect the distribution of the manipulability that are lower than 0.5. For example, in the example where the percentage of $\omega \leq 0.5$ is 16.64%, it tells us that more searching nodes are with manipulability closer to 0. Therefore, 22.39% of $\Delta\epsilon$ are in $[0.01, 0.012]$ m to generate finer step sizes. To further address the advantages of using the proposed adaptive manipulability-based RRT* method, comparisons with other RRT-based methods that are most relevant to our method have been carried out, including the adaptive RRT method [18], and manipulability-based RRT* method [21]. Table I summarizes the comparison results in terms of manipulability measure of the generated paths and searching efficiency. In [18], since the adaptive step size is considered, a higher efficiency can be obtained, i.e., $E_s = 3.124$. However, as the manipulability cost has not been considered in [18], manipulability of the generated path is lower (i.e., the average manipulability is 0.689 and the minimum 0.454), compared to that using the method in [21] or our method. Using the manipulability-based RRT* method in [21] where RRT-based method uses the manipulability cost allows higher manipulability (i.e., the average manipulability is 0.933 and the minimum 0.802). However, using the constant step size leads to the lower searching efficiency, i.e., $E_s = 1.927$. Compared to [18] and [21], our proposed adaptive manipulability-based RRT* method can obtain both the higher searching efficiency ($E_s = 3.127$) and the higher manipulability of the generated

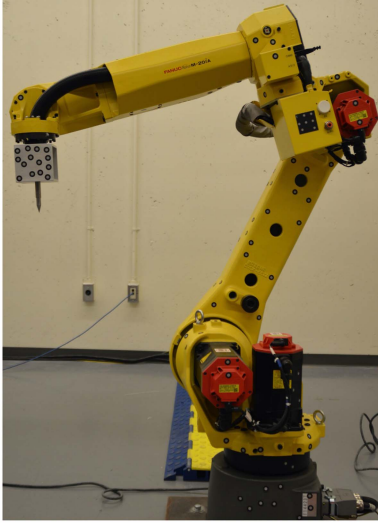


Fig. 9. FANUC-M-20iA industrial robot for experimental testing.

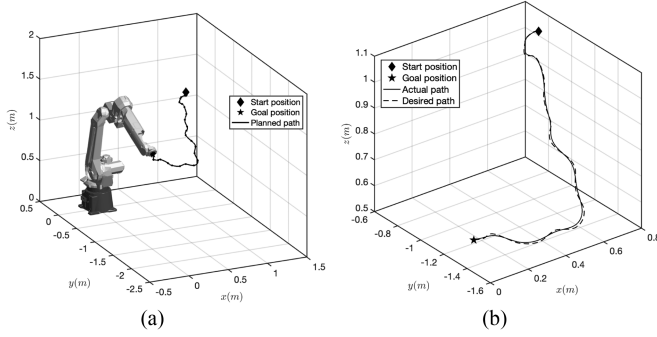


Fig. 10. (a) Planned end-effector trajectory for the FANUC robot. (b) Actual end-effector path of the FANUC robot arm compared to the planned path.

paths (i.e., the average manipulability is 1.094 and the minimum 0.979).

VI. EXPERIMENTS

To test the applicability of the adaptive manipulability-based RRT* path planner, the generated paths were tested by a 6-DOF FANUC-M-20iA industrial robot manipulator (see Fig. 9). The desired paths were generated off-line in the Matlab 2021a environment. At the path planning stage, the Newton–Euler equations model from [28] is still needed. However, to match the physical configuration of the robot with respect to the world frame, two modifications need to be made. First, in the Denavit–Hartenberg (DH) parameters, the offset along previous the joint axis to the common normal is given as $d = [0, 0, 0, 835, 0, 100]$ mm. Second, a correction matrix A_c is used for mapping the joint positions between the Newton–Euler model and the physical robot (denoted by $\mathbf{q}_{\text{model}}$ and $\mathbf{q}_{\text{real}}$, respectively), that is,

$$\mathbf{q}_{\text{model}} = A_c \mathbf{q}_{\text{real}}$$

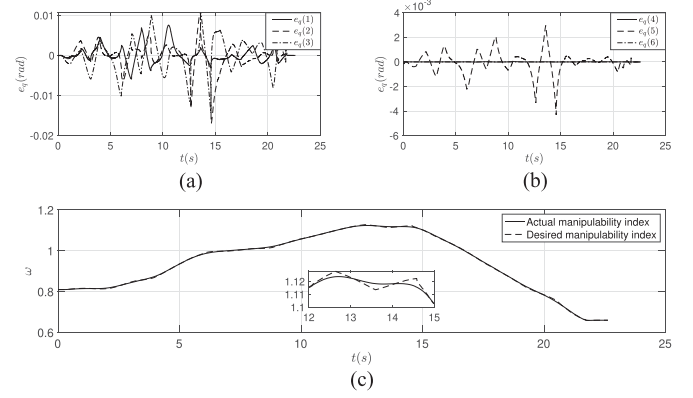


Fig. 11. Angular position tracking errors of (a) joint 1 to joint 3 and (b) joint 4 to joint 6; (c) actual manipulability profile compared to that of the planned path.

where the correction matrix $A_c \in \mathbb{R}^{6 \times 6}$ is given as

$$A_c = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix}.$$

The generated path was, then, sent to the RoboDK simulator (<https://robodk.com>), which can not only simulate the path following performance but also generate an off-line program (in “.tp” format) to run the FANUC-M-20iA industrial robot (short to FANUC robot in this article). Based on the results in Fig. 5, the desired trajectories were generated from the adaptive manipulability-based RRT* path planner by selecting $\beta = 0.2$. An example trajectory is shown in Fig. 10(a). Then, the embedded robot controller was used to drive the robot to follow the desired trajectory. Since the generated path is inherently nonsmooth, the “Termination Type” of the robot motion was set as “Continuous” (i.e., CNT100) [29] so that the robot can automatically implement a smoothed path of the original path. As a result, some small joint tracking errors can be observed, as shown in Fig. 11(a) and (b). For the same reason, in Fig. 10(b), the actual end effector path is slightly different from the desired path, especially when it comes to the sharp corners in the path. Fig. 11(c) presents that the actual path is with high manipulability as desired, which validates the feasibility of the proposed method.

VII. CONCLUSION

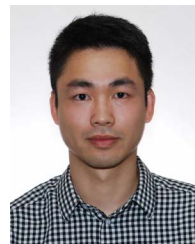
This article has presented an adaptive manipulability-based RRT* path planning strategy for industrial robot manipulators. Two constraints, path length and manipulability measure, are successfully used to find a minimal-cost path that allows the robot to traverse the workspace along short trajectories and, meanwhile, avoid configuration singularity. Additionally, the APF-based adaptive step size helps the space-filling tree to grow efficiently toward unsearched areas and converge to the

goal. Difficulties of acquiring the gradient of artificial potential have been well addressed. Numerical simulation and experiment results of a 6-DOF FANUC-M-20iA industrial robot have illustrated the efficiency of the proposed methods.

Nevertheless, to further improve the performance of the proposed methods, several problems may need to be taken into account in the future work. First, the trajectories generated from the RRT* method are inherently nonsmooth (see Fig. 10) due to the finite vertex number. Therefore, a path smoothing technique may be appreciated. Second, the path planning strategy is implemented off-line. Thus, the proposed path planning strategy works better in the scenario where the start position and the goal can be predetermined. For applications that require on-line implementation or dynamic tasks, some data training techniques (e.g., machine learning algorithms) may be promising for a real-time path planning strategy.

REFERENCES

- [1] Y. Liu, C. Guo, and M. J. Er, "Robotic 3-D laser-guided approach for efficient cutting of porcine belly," *IEEE/ASME Trans. Mechatronics*, vol. 27, no. 5, pp. 2963–2972, Oct. 2022.
- [2] P. Acharya, K.-D. Nguyen, H. M. La, D. Liu, and I.-M. Chen, "Nonprehensile manipulation: a trajectory-planning perspective," *IEEE/ASME Trans. Mechatronics*, vol. 26, no. 1, pp. 527–538, Feb. 2021.
- [3] T. T. Mac, C. Copot, D. T. Tran, and R. De Keyser, "Heuristic approaches in robot path planning: A survey," *Robot. Auton. Syst.*, vol. 86, pp. 13–28, 2016.
- [4] S. M. LaValle et al., "Rapidly-exploring random trees: A new tool for path planning," *Comput. Sci. Dept., Iowa State Univ., Ames, IA, USA*, TR 9811, 1998.
- [5] S. M. LaValle and J. J. Kuffner Jr., "Randomized kinodynamic planning," *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, 2001.
- [6] D. Bertram, J. Kuffner, R. Dillmann, and T. Asfour, "An integrated approach to inverse kinematics and path planning for redundant manipulators," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2006, pp. 1874–1879.
- [7] M. V. Weghe, D. Ferguson, and S. S. Srinivasa, "Randomized path planning for redundant manipulators without inverse kinematics," in *Proc. 7th IEEE-RAS Int. Conf. Humanoid Robots*, 2007, pp. 477–482.
- [8] S. Karaman and E. Frazzoli, "Incremental sampling-based algorithms for optimal motion planning," in *Proc. Robot. Sci. Syst. VI Conf.*, vol. 104, no. 2, 2010, pp. 1–8.
- [9] M. Otte and N. Correll, "C-Forest: Parallel shortest path planning with superlinear speedup," *IEEE Trans. Robot.*, vol. 29, no. 3, pp. 798–806, Jun. 2013.
- [10] Y. Zhang and S. Li, "Distributed biased min-consensus with applications to shortest path planning," *IEEE Trans. Autom. Control*, vol. 62, no. 10, pp. 5429–5436, Oct. 2017.
- [11] L. Zhang and C. Zhou, "Kuka youbot arm shortest path planning based on geodesics," in *Proc. IEEE Int. Conf. Robot. Biomimetics*, 2013, pp. 2317–2321.
- [12] T. Yoshikawa, "Manipulability of robotic mechanisms," *Int. J. Robot. Res.*, vol. 4, no. 2, pp. 3–9, 1985.
- [13] F. Marić, O. Limoyo, L. Petrović, T. Ablett, I. Petrović, and J. Kelly, "Fast manipulability maximization using continuous-time trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 8258–8264.
- [14] N. Vahrenkamp, T. Asfour, G. Metta, G. Sandini, and R. Dillmann, "Manipulability analysis," in *Proc. 12th IEEE-RAS Int. Conf. Humanoid Robots*, 2012, pp. 568–573.
- [15] C. Xia, Y. Zhang, and I.-M. Chen, "Learning sampling distribution for motion planning with local reconstruction-based self-organizing incremental neural network," *Neural Comput. Appl.*, vol. 31, no. 12, pp. 9185–9205, 2019.
- [16] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 7087–7094.
- [17] J. P. Wilson, Z. Shen, and S. Gupta, "A non-uniform sampling approach for fast and efficient path planning," in *Proc. OCEANS 2021: San Diego—Porto*, 2021, pp. 1–5.
- [18] C. Wang and M. Q.-H. Meng, "Variant step size RRT: An efficient path planner for UAV in complex environments," in *Proc. IEEE Int. Conf. Real-Time Comput. Robot.*, 2016, pp. 555–560.
- [19] B. An, J. Kim, and F. C. Park, "An adaptive stepsize RRT planning algorithm for open-chain robots," *IEEE Robot. Automat. Lett.*, vol. 3, no. 1, pp. 312–319, Jan. 2018.
- [20] P. Leven and S. Hutchinson, "Using manipulability to bias sampling during the construction of probabilistic roadmaps," *IEEE Trans. Robot. Automat.*, vol. 19, no. 6, pp. 1020–1026, Dec. 2003.
- [21] T. Pardi, V. Ortenzi, C. Fairbairn, T. Pipe, A. M. G. Esfahani, and R. Stolkin, "Planning maximum-manipulability cutting paths," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 1999–2006, Apr. 2020.
- [22] R. Xu, J. Luo, and M. Wang, "Optimal grasping pose for dual-arm space robot cooperative manipulation based on global manipulability," *Acta Astronautica*, vol. 183, pp. 300–309, 2021.
- [23] B. D. Luders, S. Karaman, and J. P. How, "Robust sampling-based motion planning with asymptotic optimality guarantees," in *Proc. AIAA Guid., Navigation, Control Conf.*, 2013, Art. no. 5097.
- [24] C. C. de Wit, B. Siciliano, and G. Bastin, *Theory of Robot Control*. Berlin, Germany: Springer, 2012.
- [25] J. Haviland and P. Corke, "A purely-reactive manipulability-maximising motion controller," 2020, *arXiv:2002.11901*.
- [26] J. Haviland and P. Corke, "A systematic approach to computing the manipulator Jacobian and Hessian using the elementary transform sequence," 2020, *arXiv:2010.08696*.
- [27] J. Nakanishi, R. Cory, M. Mistry, J. Peters, and S. Schaal, "Comparative experiments on task space control with redundancy resolution," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2005, pp. 3901–3908.
- [28] Y. Zhao, H.-C. Lin, and M. Tomizuka, "Efficient trajectory optimization for robot motion planning," in *Proc. 15th Int. Conf. Control, Automat., Robot. Vis.*, 2018, pp. 260–265.
- [29] R. R. Garcia, A. C. Bittencourt, and E. Villani, "Relevant factors for the energy consumption of industrial robots," *J. Braz. Soc. Mech. Sci. Eng.*, vol. 40, no. 9, 2018, Art. no. 464.



Henghua Shen (Member, IEEE) received the B.E. degree in mechanical engineering and the M.E. degree in mechanical engineering from Fuzhou University, Fuzhou, China, in 2012 and 2015, respectively, and the Ph.D degree in mechanical engineering from Dalhousie University, Halifax, NS, Canada, in 2020.

He is currently a Horizon Postdoctoral Fellow with the Department of Mechanical, Industrial, and Aerospace Engineering, Concordia University, Montréal, QC, Canada. His research interests include computer vision, robotics and networked control systems.

Dr. Shen is a Member of IEEE Control Systems Society and IEEE Industrial Electronics Society.



Wen-Fang Xie (Senior Member, IEEE) received the master's degree in automatic control from the Beijing University of Aeronautics and Astronautics, Beijing, China, in 1991, and the Ph.D. degree in electrical engineering from Hong Kong Polytechnic University, Hong Kong, in 1999.

She served as a Senior Research Engineer with InCoreTec, Inc., St. John's, NL, Canada, from 2002 to 2003. She joined Concordia University, Montreal, QC, Canada, in 2003. She is currently a Professor with the Department of Mechanical, Industrial and Aerospace Engineering, Concordia University. Her research interests include nonlinear control and identification in mechatronics, artificial intelligent control, visual servoing and robotics.

Dr. Xie held an Industrial Research Fellowship from the Natural Sciences and Engineering Research Council of Canada from January 2002 to September 2003.



Jianyu Tang received the B.E. degree in mechanical engineering from Nantong University, Nantong, China, in 2019, and the M.A.Sc. degree in mechanical engineering from Concordia University, Montreal, QC, Canada, in 2022.

He is the Lab Manager of Dr. Wen-Fang Xie's research group with expertise in the operation and safety of FANUC robots. His research interests include robotic control systems, path planning, visual servoing, automation, intelligent manufacturing, and neural networks.



Tao Zhou received the B.E. degree in mechanical engineering from McGill University, Montreal, QC, Canada, in 2019, and the M.A.Sc. degree in mechanical engineering from Concordia University, Montreal, QC, Canada, in 2022.

He is highly proficient in the programming of FANUC robots. His research interests include robotics, control systems, reinforcement learning, visual servoing, and industrial automation.

Mr. Zhou was the recipient of Canada Graduate Scholarships-Master (NSERC).